Phala Network 팰릿 구현 분석

1. Phala Pallets 개요

Phala Pallets는 **Phala Network의 핵심 비즈니스 로직을 구현하는 Substrate 팰릿 모음**으로, TEE 워커 관리, 클러스터 운영, 스마트 컨트랙트 배포 등 Phala 생태계의 모든 온체인 기능을 담당합니다.

2. 팰릿 구조 분석

2.1 핵심 의존성

```
# Substrate 프레임워크
├── frame-support: 팰릿 개발 프레임워크
├── frame-system: 시스템 기본 기능
├── sp-runtime: 런타임 프리미티브
├── sp-std: no_std 표준 라이브러리

# 외부 팰릿 통합
├── pallet-balances: 잔액 관리
├── pallet-assets: 자산 관리
├── pallet-democracy: 거버넌스
├── pallet-uniques: NFT 관리
```

```
# Phala 특화 컴포넌트
├── phala-types: Phala 타입 정의
├── sgx-attestation: SGX 증명 검증
├── pallet-rmrk-core: RMRK NFT 지원
└── rmrk-traits: RMRK 트레이트
```

2.2 팰릿 모듈 구조

```rust
// 팰릿 구조 예시 (추정)
pub mod pallets {
pub mod registry; // 워커/클러스터 등록 관리
pub mod computation; // 컴퓨팅 작업 관리
pub mod stakepool; // 스테이크 풀 관리
pub mod mining; // 마이닝 보상 시스템
pub mod fat_contract; // Fat Contract 관리
pub mod pink; // Pink 런타임 통합
pub mod mq; // 메시지 큐 시스템
pub mod utils; // 유틸리티 함수들
}
```

3. Registry 팰릿 (워커 관리)

3.1 워커 등록 시스템

```rust
// 워커 정보 구조체 (추정)
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct WorkerInfo<AccountId, Balance> {
// 워커 소유자
pub owner: AccountId,
// 워커 공개키
pub pubkey: WorkerPublicKey,
// SGX Attestation 정보
pub attestation: Option<Attestation>,
// 워커 상태
pub state: WorkerState,
// 스테이킹된 금액
pub stake: Balance,
// 등록 시점
pub registered_at: BlockNumber,
// 마지막 활동 시점
pub last_updated: BlockNumber,
// 성능 점수
pub confidence_level: u8,
}

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
```

```
pub enum WorkerState {
// 등록됨 (아직 증명 안됨)
Registered,
// 증명 완료 (활성 상태)
Ready,
// 작업 중
Working,
// 일시 중지
Paused,
// 제거됨
Removed,
}
```

3.2 워커 등록 로직

```
// 워커 등록 구현 (추정)
impl<T: Config> Pallet<T> {
#[pallet::call_index(0)]
#[pallet::weight(T::WeightInfo::register_worker())]
pub fn register_worker(
origin: OriginFor<T>,
worker_pubkey: WorkerPublicKey,
attestation_payload: Vec<u8>,
```

```
attestation_signature: Vec<u8>,

) -> DispatchResult {

let  who = ensure_signed(origin)?;

// 1. 워커 공개키 중복 확인

ensure!(

!Workers::<T>::contains_key(&worker_pubkey),

Error::<T>::WorkerAlreadyExists

);

// 2. SGX Attestation 검증

let  attestation = Self::verify_attestation(

&worker_pubkey,

&attestation_payload,

&attestation_signature,

)?;

// 3. 최소 스테이킹 확인

let  stake_amount = T::MinWorkerStake::get();

T::Currency::reserve(&who, stake_amount)?;

// 4. 워커 정보 생성

let  worker_info = WorkerInfo {

owner: who.clone(),

pubkey: worker_pubkey.clone(),

attestation: Some(attestation),

state: WorkerState::Ready,
```

```
stake: stake_amount,
registered_at: frame_system::Pallet::<T>::block_number(),
last_updated: frame_system::Pallet::<T>::block_number(),
confidence_level: 100u8, // 초기 신뢰도 100%
};
// 5. 스토리지 업데이트
Workers::<T>::insert(&worker_pubkey, &worker_info);
WorkersByOwner::<T>::append(&who, &worker_pubkey);
// 6. 이벤트 발생
Self::deposit_event(Event::WorkerRegistered {
worker: worker_pubkey,
owner: who,
initial_stake: stake_amount,
});
Ok(())
}
fn verify_attestation(
worker_pubkey: &WorkerPublicKey,
payload: &[u8],
signature: &[u8],
) -> Result<Attestation, Error<T>> {
// 1. 서명 검증
let is_valid = sgx_attestation::verify_signature(
```

```
worker_pubkey.as_ref(),
payload,
signature,
).map_err(|_| Error::<T>::InvalidAttestationSignature)?;
ensure!(is_valid, Error::<T>::InvalidAttestationSignature);
// 2. Attestation 파싱
let  attestation = sgx_attestation::parse_attestation(payload)
.map_err(|_| Error::<T>::InvalidAttestationFormat)?;
// 3. TCB 상태 확인
ensure!(
attestation.tcb_status == TcbStatus::UpToDate,
Error::<T>::OutdatedTcb
);
// 4. MRENCLAVE 검증
let  expected_mrenclave = T::ExpectedMrenclave::get();
ensure!(
attestation.mr_enclave == expected_mrenclave,
Error::<T>::UnexpectedMrenclave
);
Ok(attestation)
}
}
```

4. Computation 팰릿 (컴퓨팅 작업)

4.1 클러스터 관리

```
// 클러스터 정보 구조체 (추정)
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct ClusterInfo<AccountId> {
// 클러스터 소유자
pub owner: AccountId,
// 클러스터에 속한 워커들
pub workers: BoundedVec<WorkerPublicKey, T::MaxWorkersPerCluster>,
// 클러스터 상태
pub state: ClusterState,
// 생성 시점
pub created_at: BlockNumber,
// 총 컴퓨팅 파워
pub total_power: u64,
// 활성 컨트랙트 수
pub active_contracts: u32,
}

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub enum ClusterState {
```

```
// 생성됨
Created,
// 활성 상태
Active,
// 유지보수 모드
Maintenance,
// 비활성화됨
Inactive,
}
```

4.2 컨트랙트 배포

```
// 컨트랙트 배포 구현 (추정)
impl<T: Config> Pallet<T> {
#[pallet::call_index(10)]
#[pallet::weight(T::WeightInfo::deploy_contract())]
pub fn deploy_contract(
origin: OriginFor<T>,
cluster_id: ClusterId,
contract_code: Vec<u8>,
constructor_data: Vec<u8>,
salt: Vec<u8>,
deposit: BalanceOf<T>,
```

```
) -> DispatchResult {
let who = ensure_signed(origin)?;
// 1. 클러스터 존재 확인
let cluster = Clusters::<T>::get(&cluster_id)
.ok_or(Error::<T>::ClusterNotFound)?;
// 2. 클러스터 상태 확인
ensure!(
cluster.state == ClusterState::Active,
Error::<T>::ClusterNotActive
);
// 3. 컨트랙트 ID 생성
let contract_id = Self::generate_contract_id(&who, &contract_code, &salt)?;
// 4. 중복 확인
ensure!(
!Contracts::<T>::contains_key(&contract_id),
Error::<T>::ContractAlreadyExists
);
// 5. 보증금 예치
T::Currency::reserve(&who, deposit)?;
// 6. 컨트랙트 정보 생성
let contract_info = ContractInfo {
deployer: who.clone(),
cluster_id: cluster_id.clone(),
```

```
code_hash: Self::hash_contract_code(&contract_code),

state: ContractState::Deploying,

deployed_at: frame_system::Pallet::<T>::block_number(),

deposit,

};

// 7. 스토리지 업데이트

Contracts::<T>::insert(&contract_id, &contract_info);

ContractsByCluster::<T>::append(&cluster_id, &contract_id);

// 8. 배포 메시지 큐에 추가

Self::queue_deployment_message(contract_id.clone(), contract_code, constructor_d

// 9. 이벤트 발생

Self::deposit_event(Event::ContractDeployed {

contract_id,

deployer: who,

cluster_id,

});

Ok(())

}

}
```

5. StakePool 팰릿 (스테이킹 풀)

5.1 스테이킹 풀 구조

```rust
// 스테이킹 풀 정보 (추정)
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct StakePoolInfo<AccountId, Balance> {
// 풀 소유자 (워커)
pub owner: AccountId,
// 연결된 워커
pub worker: WorkerPublicKey,
// 총 스테이킹된 금액
pub total_stake: Balance,
// 풀 수수료 (퍼센트)
pub commission: Percent,
// 활성 상태
pub is_active: bool,
// 생성 시점
pub created_at: BlockNumber,
// 마지막 보상 분배 시점
pub last_reward_distribution: BlockNumber,
}

// 스테이커 정보
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct StakerInfo<Balance> {
```

```
// 스테이킹된 금액
pub staked_amount: Balance,
// 스테이킹 시점
pub staked_at: BlockNumber,
// 누적 보상
pub accumulated_rewards: Balance,
// 마지막 보상 청구 시점
pub last_reward_claim: BlockNumber,
}
```

5.2 스테이킹 로직

```
// 스테이킹 구현 (추정)
impl<T: Config> Pallet<T> {
#[pallet::call_index(20)]
#[pallet::weight(T::WeightInfo::stake_to_pool())]
pub fn stake_to_pool(
origin: OriginFor<T>,
pool_id: PoolId,
amount: BalanceOf<T>,
) -> DispatchResult {
let who = ensure_signed(origin)?;
// 1. 풀 존재 확인
```

```
let mut pool_info = StakePools::<T>::get(&pool_id)
.ok_or(Error::<T>::PoolNotFound)?;
// 2. 풀 활성 상태 확인
ensure!(pool_info.is_active, Error::<T>::PoolNotActive);
// 3. 최소 스테이킹 금액 확인
ensure!(
amount >= T::MinStakeAmount::get(),
Error::<T>::InsufficientStakeAmount
);
// 4. 잔액 확인 및 예치
T::Currency::reserve(&who, amount)?;
// 5. 기존 스테이커 정보 조회 또는 생성
let mut staker_info = Stakers::<T>::get(&pool_id, &who)
.unwrap_or_default();
// 6. 보상 계산 및 업데이트
let pending_rewards = Self::calculate_pending_rewards(&pool_id, &who)?;
staker_info.accumulated_rewards = staker_info.accumulated_rewards
.saturating_add(pending_rewards);
// 7. 스테이킹 정보 업데이트
staker_info.staked_amount = staker_info.staked_amount.saturating_add(amount);
staker_info.last_reward_claim = frame_system::Pallet::<T>::block_number();
if staker_info.staked_at == 0u32.into() {
staker_info.staked_at = frame_system::Pallet::<T>::block_number();
```

```
}

// 8. 풀 총 스테이킹 업데이트

pool_info.total_stake = pool_info.total_stake.saturating_add(amount);

// 9. 스토리지 업데이트

Stakers::<T>::insert(&pool_id, &who, &staker_info);

StakePools::<T>::insert(&pool_id, &pool_info);

// 10. 이벤트 발생

Self::deposit_event(Event::Staked {

pool_id,

staker: who,

amount,

});

Ok(())

}

fn  calculate_pending_rewards(

pool_id: &PoolId,

staker: &T::AccountId,

) -> Result<BalanceOf<T>, Error<T>> {

// 보상 계산 로직 구현

// 1. 마지막 보상 청구 이후 블록 수 계산

// 2. 풀의 총 보상률 적용

// 3. 스테이커의 지분 비율 계산

// 4. 최종 보상 금액 반환
```

```
Ok(Zero::zero()) // 간소화된 구현
}
}
```

6. Mining 팰릿 (마이닝 보상)

6.1 마이닝 세션 관리

```
// 마이닝 세션 정보 (추정)
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct MiningSession<BlockNumber> {
// 세션 시작 블록
pub start_block: BlockNumber,
// 세션 종료 블록
pub end_block: BlockNumber,
// 총 보상 풀
pub total_reward_pool: Balance,
// 참여 워커 수
pub participating_workers: u32,
// 세션 상태
pub state: SessionState,
}
```

```
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub enum SessionState {
Active,
Ended,
RewardsDistributed,
}
```

6.2 보상 분배 로직

```
// 보상 분배 구현 (추정)
impl<T: Config> Pallet<T> {
#[pallet::call_index(30)]
#[pallet::weight(T::WeightInfo::distribute_rewards())]
pub fn distribute_rewards(
origin: OriginFor<T>,
session_id: SessionId,
) -> DispatchResult {
ensure_root(origin)?;
// 1. 세션 정보 조회
let mut session = MiningSessions::<T>::get(&session_id)
.ok_or(Error::<T>::SessionNotFound)?;
// 2. 세션 상태 확인
ensure!(
```

```
session.state == SessionState::Ended,
Error::<T>::SessionNotEnded
);
// 3. 워커별 기여도 계산
let worker_contributions = Self::calculate_worker_contributions(&session_id)?;
// 4. 보상 분배
for (worker_pubkey, contribution) in worker_contributions {
let reward_amount = session.total_reward_pool
.saturating_mul(contribution.into())
.saturating_div(100u32.into()); // contribution은 퍼센트
// 워커 소유자에게 보상 지급
if let Some(worker_info) = Workers::<T>::get(&worker_pubkey) {
let _ = T::Currency::deposit_creating(&worker_info.owner, reward_amount);
// 스테이킹 풀이 있다면 풀에도 보상 분배
if let Some(pool_id) = WorkerPools::<T>::get(&worker_pubkey) {
Self::distribute_pool_rewards(&pool_id, reward_amount)?;
}
}
}
// 5. 세션 상태 업데이트
session.state = SessionState::RewardsDistributed;
MiningSessions::<T>::insert(&session_id, &session);
// 6. 이벤트 발생
```

```
Self::deposit_event(Event::RewardsDistributed {
session_id,
total_amount: session.total_reward_pool,
});
Ok(())
}
}
```

7. 메시지 큐 시스템

7.1 메시지 타입 정의

```
// 메시지 타입 (추정)
#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub  enum MessageOrigin {
// 사용자 트랜잭션에서 발생
User(AccountId),
// 시스템에서 자동 생성
System,
// 워커에서 전송
Worker(WorkerPublicKey),
// 클러스터에서 전송
Cluster(ClusterId),
```

```
}

#[derive(Encode, Decode, Clone, PartialEq, Eq, RuntimeDebug, TypeInfo)]
pub struct Message {
// 메시지 ID
pub id: MessageId,
// 발신자
pub origin: MessageOrigin,
// 수신자 (워커 또는 클러스터)
pub destination: MessageDestination,
// 메시지 페이로드
pub payload: Vec<u8>,
// 생성 시점
pub created_at: BlockNumber,
// 만료 시점
pub expires_at: BlockNumber,
}
```

7.2 메시지 처리

```
// 메시지 큐 구현 (추정)
impl<T: Config> Pallet<T> {
pub fn queue_message(
```

```
origin: MessageOrigin,

destination: MessageDestination,

payload: Vec<u8>,

) -> Result<MessageId, Error<T>> {

// 1. 메시지 ID 생성

let  message_id = Self::next_message_id();

// 2. 메시지 생성

let  message = Message {

id: message_id,

origin,

destination: destination.clone(),

payload,

created_at: frame_system::Pallet::<T>::block_number(),

expires_at: frame_system::Pallet::<T>::block_number() + T::MessageTtl::get(),

};

// 3. 큐에 추가

MessageQueue::<T>::append(&destination, &message);

Messages::<T>::insert(&message_id, &message);

// 4. 메시지 카운터 증가

let  next_id = NextMessageId::<T>::get().saturating_add(1);

NextMessageId::<T>::put(next_id);

Ok(message_id)

}
```

```
pub fn dequeue_messages(destination: &MessageDestination) -> Vec<Message> {
let messages = MessageQueue::<T>::get(destination);
// 만료된 메시지 필터링
let current_block = frame_system::Pallet::<T>::block_number();
let valid_messages: Vec<Message> = messages
.into_iter()
.filter(|msg| msg.expires_at > current_block)
.collect();
// 큐 업데이트
MessageQueue::<T>::insert(destination, &valid_messages);
valid_messages
}
}
```

8. 이벤트 및 에러 정의

8.1 팰릿 이벤트

```
// 이벤트 정의 (추정)
#[pallet::event]
#[pallet::generate_deposit(pub(super) fn deposit_event)]
pub enum Event<T: Config> {
/// 워커가 등록됨
```

```
WorkerRegistered {
worker: WorkerPublicKey,
owner: T::AccountId,
initial_stake: BalanceOf<T>,
},
/// 클러스터가 생성됨
ClusterCreated {
cluster_id: ClusterId,
owner: T::AccountId,
},
/// 컨트랙트가 배포됨
ContractDeployed {
contract_id: ContractId,
deployer: T::AccountId,
cluster_id: ClusterId,
},
/// 스테이킹됨
Staked {
pool_id: PoolId,
staker: T::AccountId,
amount: BalanceOf<T>,
},
/// 보상이 분배됨
```

```
RewardsDistributed {
session_id: SessionId,
total_amount: BalanceOf<T>,
},
}
```

8.2 에러 정의

```
// 에러 정의 (추정)
#[pallet::error]
pub  enum Error<T> {
/// 워커가 이미 존재함
WorkerAlreadyExists,
/// 워커를 찾을 수 없음
WorkerNotFound,
/// 잘못된 SGX Attestation
InvalidAttestation,
/// 클러스터를 찾을 수 없음
ClusterNotFound,
/// 클러스터가 활성 상태가 아님
ClusterNotActive,
/// 컨트랙트가 이미 존재함
ContractAlreadyExists,
```

```
/// 스테이킹 풀을 찾을 수 없음

PoolNotFound,

/// 스테이킹 금액이 부족함

InsufficientStakeAmount,

/// 권한이 없음

NoPermission,

/// 잔액이 부족함

InsufficientBalance,

}
```

이 문서는 Phala Network의 핵심 팰릿 구현을 다룹니다. 다음 문서에서는 프론트엔드 구현을 분석합니다.